

PROJET DE FIN D'ÉTUDES · 2025 / 2026

Dossier de Conception & Plan de Réalisation

StockFlow

Application web de gestion de stock multi-entrepôts

Chef de projet : Amine

Équipe : Hiba · Ayoub · Othman · Youssef · Abdelhamid

Établissement : HighTech Rabat

Période : 11 → 30 juin 2026

Stack : Laravel 12 · React 18 · PostgreSQL · Sanctum · Tailwind CSS

Contenu : conception base de données · backend · frontend · API REST · rôles · répartition des tâches · diagramme de Gantt.

Table des matières

1. Vue d'ensemble de la conception	2
2. Conception de la base de données	3
3. Conception du backend (Laravel 12)	5
4. Conception du frontend (React + Vite)	6
5. Répartition des tâches de l'équipe	7
6. Diagramme de Gantt	9

1. Vue d'ensemble de la conception

StockFlow est une application web de gestion de stock multi-entrepôts dédiée à une seule entreprise, hébergée en ligne. La conception repose sur une séparation stricte frontend (React 18 + Vite + Tailwind + shadcn/ui) / backend (Laravel 12 + Sanctum + Spatie Permission) / base de données (PostgreSQL), avec une couche de **Services métier** qui concentre les règles de gestion dans des transactions atomiques.

Couche	Technologies	Responsabilité
Frontend	React 18, Vite, TypeScript, Tailwind CSS, shadcn/ui, Recharts, Axios, React Router	SPA, interfaces, graphiques, consommation API.
Backend	Laravel 12, Sanctum, Spatie Permission, Activity Log, DomPDF, Laravel Excel	API REST /api/v1, règles métier, sécurité, exports.
Base de données	PostgreSQL (Neon)	Persistance, contraintes d'intégrité, colonnes générées.

Principes structurants : (1) le `disponible` est une colonne **générée** (`quantite - reserve`), jamais saisie ; (2) tout mouvement passe par `StockService` en **transaction + verrou pessimiste** ; (3) un transfert = machine à états avec double mouvement (sortie à la validation, entrée à la réception) ; (4) RBAC à 4 rôles via Spatie.

2. Conception de la base de données

users — utilisateurs de l'application

Colonne	Type	Contraintes	Description
id	uuid	PK	Identifiant unique
nom / prenom	varchar(80)	NN	Identité
email	varchar(150)	UQ NN	Identifiant de connexion
password	varchar	NN	Hash bcrypt/argon2
telephone	varchar(20)		Nullable
actif	boolean	NN	Défaut true — désactivation sans suppression
timestamps	timestamp		created_at / updated_at

warehouses — entrepôts de l'entreprise

Colonne	Type	Contraintes	Description
id	uuid	PK	
nom	varchar(100)	NN	Libellé
code	varchar(20)	UQ NN	Code court (ex. WH-CAS-01)
adresse	text		Adresse physique
responsable / telephone	varchar		Nullable
actif	boolean	NN	Défaut true

categories — catégories de produits

Colonne	Type	Contraintes	Description
id	uuid	PK	
nom	varchar(100)	UQ NN	Nom de catégorie
description	text		Nullable

products — catalogue de produits

Colonne	Type	Contraintes	Description
id	uuid	PK	
categorie_id	uuid	FK NN	→ categories.id
sku	varchar(50)	UQ NN	Référence interne unique
code_barre	varchar(50)	UQ	EAN/QR — nullable
nom	varchar(150)	NN	Libellé
description	text		Nullable
prix_achat / prix_vente	decimal(10,2)	NN	≥ 0, défaut 0
stock_minimum	integer	NN	Seuil d'alerte, défaut 0
image	varchar		Chemin/URL — nullable
actif	boolean	NN	Défaut true

stocks — niveaux de stock par couple produit/entrepôt

Colonne	Type	Contraintes	Description
id	uuid	PK	
product_id	uuid	FK NN	→ products.id
warehouse_id	uuid	FK NN	→ warehouses.id
quantite	integer	NN CHECK ≥ 0	Stock physique
reserve	integer	NN CHECK ≥ 0	Réservé (transferts validés en attente de réception)
disponible	integer	GENERATED	quantite - reserve — jamais saisi

Contrainte clé : UNIQUE(product_id, warehouse_id) — une seule ligne de stock par couple produit/entrepôt.

stock_movements — historique immuable de toutes les variations

Colonne	Type	Contraintes	Description
id	uuid	PK	
reference	varchar(30)	UQ NN	MVT-2026-000001 — auto-générée
type	enum	NN	achat, retour_fournisseur, ajustement_entree, vente, consommation, perte, ajustement_sortie, transfert_sortie, transfert_entree
quantite	integer	NN CHECK > 0	Toujours positive (le type porte le sens)
product_id / warehouse_id / user_id	uuid	FK NN	Produit, entrepôt, auteur
transfer_id	uuid	FK	Nullable — lien vers le transfert d'origine
motif	text		Commentaire — nullable
created_at	timestamp	NN	Pas de updated_at : enregistrement immuable

transfers — transferts inter-entrepôts

Colonne	Type	Contraintes	Description
id	uuid	PK	
reference	varchar(30)	UQ NN	TRF-2026-000001 — auto-générée
source_warehouse_id	uuid	FK NN	CHECK source ≠ destination
dest_warehouse_id	uuid	FK NN	→ warehouses.id
statut	enum	NN	brouillon → en_attente → valide → reçu annule
created_by / validated_by / received_by	uuid	FK	Traçabilité par étape (validated/received nullables)
validated_at / received_at	timestamp		Nullables
note	text		Nullable

transfer_items — lignes de produits d'un transfert

Colonne	Type	Contraintes	Description
id	uuid	PK	
transfer_id / product_id	uuid	FK NN	UNIQUE(transfer_id, product_id)
quantite	integer	NN CHECK > 0	

inventories — sessions d'inventaire

Colonne	Type	Contraintes	Description
id	uuid	PK	
warehouse_id / user_id	uuid	FK NN	Entrepôt inventorié + créateur
type	enum	NN	global tournant
statut	enum	NN	en_cours cloture ajuste
started_at / closed_at	timestamp		closed_at nullable

inventory_items — lignes de comptage

Colonne	Type	Contraintes	Description
id	uuid	PK	
inventory_id / product_id	uuid	FK NN	UNIQUE(inventory_id, product_id)
qte_theorique	integer	NN	Copie du stock au démarrage (snapshot)
qte_reelle	integer		Null = pas encore compté
ecart	integer	GENERATED	qte_reelle - qte_theorique

Tables système (générées par les packages)

roles , permissions , model_has_roles , model_has_permissions , role_has_permissions (Spatie Permission) · activity_log (Spatie Activity Log : subject, causer, event, properties JSON old/new) · personal_access_tokens (Sanctum).

3. Conception du backend (Laravel 12)

3.1 Structure des dossiers

```
app/
├── Models/                # User, Warehouse, Category, Product, Stock,
│                           # StockMovement, Transfer, TransferItem,
│                           # Inventory, InventoryItem
├── Http/Controllers/Api/V1/
│   ├── AuthController    # login / logout / me
│   ├── UserController    # CRUD + assignation rôle (admin)
│   ├── WarehouseController # CRUD + soft delete
│   ├── CategoryController # CRUD
│   ├── ProductController # CRUD + upload image
│   ├── StockController   # index (filtres) + alertes
│   ├── MovementController # index + store (→ StockService)
│   ├── TransferController # CRUD + submit/validate/receive/cancel
│   ├── InventoryController # open / saisie / close / adjust
│   ├── DashboardController # KPIs + séries graphiques
│   └── ReportController   # PDF / Excel selon ?format=
├── Http/Requests/        # Form Requests de validation
├── Http/Resources/       # transformateurs JSON
├── Http/Middleware/      # CheckRole (403 si rôle insuffisant)
├── Policies/             # autorisations par ressource
├── Services/
│   ├── StockService       # createMovement() – transaction + verrou
│   ├── TransferService    # validate() / receive() – machine à états
│   ├── InventoryService   # open() / adjust() – mouvements d'écart
│   └── ReportService       # generatePDF() / generateExcel()
├── Enums/                # MovementType, TransferStatus
├── database/
│   ├── migrations/       # ordre FK : warehouses → categories → products
│   │                       # → stocks → transfers → movements → inventories
│   ├── seeders/         # RolePermissionSeeder + DemoSeeder
│   └── factories/        # pour les tests Pest
├── routes/api.php        # groupes : public / auth:sanctum / role:admin...
```

3.2 Logique critique — StockService

```
// Tout mouvement de stock passe par cette méthode unique
DB::transaction(function () use ($data) {
    $stock = Stock::where([
        'product_id' => $data['product_id'],
        'warehouse_id' => $data['warehouse_id'],
    ])->lockForUpdate()->firstOrFail(); // verrou pessimiste

    if (MovementType::isSortie($data['type'])) {
        if ($stock->disponible < $data['quantite'])
            throw new InsufficientStockException(); // RG : refus
        $stock->decrement('quantite', $data['quantite']);
    } else {
        $stock->increment('quantite', $data['quantite']);
    }

    StockMovement::create($data + ['reference' => Ref::next('MVT')]);
    activity()->performedOn($stock)->log('mouvement');

    if ($stock->fresh()->disponible <= $stock->product->stock_minimum)
        event(new StockLow($stock)); // alerte seuil
});
```

3.3 Machine à états — TransferService

Transition	Déclencheur	Effet sur les stocks
brouillon → en_attente	<code>submit()</code> — Magasinier	Aucun
en_attente → valide	<code>validate()</code> — Gestionnaire	Sortie de la source (<code>transfert_sortie</code>) + <code>reserve</code> incrémenté
valide → reçu	<code>receive()</code> — Magasinier destination	Entrée en destination (<code>transfert_entree</code>) + <code>reserve</code> libéré
* → annule	<code>cancel()</code> — Gestionnaire	Si déjà validé : ré-entrée dans la source

4. Conception du frontend (React + Vite)

4.1 Structure des dossiers

```
src/
├── api/
│   ├── axios.js           # instance + intercepteur token + gestion 401
│   └── *.api.js           # un fichier par ressource (auth, products,
│                           # stocks, mouvements, transfers, inventories,
│                           # dashboard, reports)
├── components/
│   ├── ui/                # shadcn/ui : Button, Input, Dialog, Table...
│   └── shared/
│       ├── AppLayout      # sidebar + header + outlet
│       ├── Sidebar        # liens de navigation selon le rôle
│       ├── DataTable      # tableau réutilisable (tri/pagination/recherche)
│       ├── StockBadge     # vert / orange / rouge selon disponible vs seuil
│       ├── StatusBadge    # statut transfert / inventaire
│       └── ConfirmDialog  # confirmation de suppression
├── features/              # organisation par fonctionnalité
│   ├── auth/              # LoginPage + useAuth
│   ├── dashboard/        # KpiCard + 3 graphiques Recharts
│   ├── warehouses/
│   ├── categories/
│   ├── users/
│   ├── products/         # liste + modale formulaire + détail
│   ├── stocks/           # vue stock filtrée + page alertes
│   ├── mouvements/      # formulaire avec contrôle disponible temps réel
│   ├── transfers/        # wizard multi-étapes + page détail (actions)
│   ├── inventories/      # session de saisie + écarts + ajustement
│   ├── reports/          # sélection type + format → téléchargement blob
│   └── activity/         # timeline du journal filtrée
├── hooks/                 # useAuth, useApi, useDebounce
├── contexts/              # AuthContext (user + token)
├── routes/                # createBrowserRouter + ProtectedRoute (rôle)
└── lib/
    └── utils.js           # formatDate, formatMoney, cn()
```

4.2 Spécification de l'API REST (préfixe /api/v1)

Méthode	Route	Accès	Description
POST	/login · /logout	Public / Auth	Token Sanctum / révocation
GET	/me	Auth	Utilisateur + permissions
GET POST PUT	/users · /users/{id} · /users/{id}/toggle	Admin	CRUD + activation/désactivation
GET POST PUT DEL	/warehouses	Auth / Admin	Lecture pour tous, écriture admin
GET POST PUT DEL	/categories · /products	Auth / Gestionnaire	Lecture pour tous, écriture gestionnaire
GET	/stocks · /stocks/alerts	Auth	Filtres ?warehouse_id ?product_id ?alert=1
GET POST	/movements	Auth / Magasinier+	Historique / création via StockService
POST	/transfers · /{id}/submit · /{id}/validate · /{id}/receive · /{id}/cancel	Magasinier / Gestionnaire	Cycle complet du transfert
POST PUT	/inventories · /{id}/items/{itemId} · /{id}/close · /{id}/adjust	Gestionnaire / Magasinier+	Session, saisie, clôture, ajustement
GET	/dashboard	Auth	KPIs + séries (30 jours)
GET	/reports/{type}?format=pdf xlsx	Auth	stock_global, mouvements, inventaire
GET	/activity	Auditeur+	Journal d'activité paginé

4.3 Rôles et permissions (Spatie)

Rôle	Permissions
Administrateur	Tout + <code>manage-users</code> , <code>manage-warehouses</code> , <code>view-activity</code>
Gestionnaire de stock	CRUD produits/catégories · <code>transfers.validate/cancel</code> · <code>inventories.create/adjust</code> · <code>view-reports</code> , <code>export</code> , <code>view-dashboard</code>
Magasinier	<code>movements.create</code> · <code>transfers.create/receive</code> · <code>inventories.update</code> (saisie) · consultation stocks
Auditeur	Lecture seule sur tout · <code>view-activity</code> , <code>view-reports</code> , <code>export</code> , <code>view-dashboard</code> — aucune écriture

5. Répartition des tâches de l'équipe

L'équipe est composée de 6 membres. La répartition suit le principe du **découpage par module** : chaque développeur possède ses modules de bout en bout, ce qui limite les dépendances croisées. Amine, chef de projet, conserve l'architecture, la sécurité et le service de stock (code le plus critique), et supervise l'intégration.

Amine — Chef de projet & Lead technique

Architecture & setup : initialisation Laravel 12, structure des dossiers, configuration PostgreSQL (Neon), Docker, routes API groupées par middleware.

Auth & sécurité : AuthController (login/logout/me), Sanctum (tokens + middleware), Spatie Permission (RolePermissionSeeder), middleware CheckRole.

Service critique : StockService (transaction + lockForUpdate), TransferService, InventoryService, événements StockLow.

Supervision : revues de code des PR, tests d'intégration globaux, déploiement final.

Abdelhamid — Migrations, Models & Base de données

Toutes les migrations (dans l'ordre des FK) :

warehouses → categories → products → stocks → transfers + transfer_items → stock_movements → inventories + inventory_items

Tous les models Eloquent : User, Warehouse, Category, Product, Stock, StockMovement, Transfer, TransferItem, Inventory, InventoryItem — avec toutes les relations (`hasMany` , `belongsTo` , `hasOne`), les casts, les `fillable` , les scopes, et les colonnes `GENERATED` (`disponible` , `ecart`).

Seeders & factories : RolePermissionSeeder (4 rôles + permissions Spatie), DemoSeeder (entrepôts, produits, stocks de démonstration), factories pour les tests Pest.

Ayoub — Controllers groupe 1 & Routes

Controllers (entités de base) :

- UserController : liste, création, modification, activation/désactivation, assignation de rôle.
- WarehouseController : CRUD complet + soft delete (si stock = 0).
- CategoryController : CRUD.
- ProductController : CRUD + upload image (multipart/form-data).

Pour chaque controller : Form Requests (validation), API Resources (transformateurs JSON), Policy (autorisations).

Routes associées : déclaration dans `routes/api.php` du groupe `auth:sanctum` pour ses endpoints — avec les middlewares de rôle appropriés (`admin` pour users/warehouses, `gestionnaire` pour products/categories).

Tests Postman : GET liste (pagination + filtres), GET détail (404 si inexistant), POST création (cas valide 201, cas 422 avec erreurs de validation), PUT modification, DELETE/soft-delete, vérification des codes HTTP et corps JSON pour chaque endpoint.

Othman — Controllers groupe 2, Routes & Frontend

Controllers (modules métier) :

- StockController : index avec filtres (warehouse_id, product_id, alert=1), endpoint alertes.
- MovementController : liste paginée, création (délègue à StockService d'Amine).
- TransferController : CRUD + submit / validate / receive / cancel (délègue à TransferService).
- InventoryController : ouverture, saisie des quantités, clôture, ajustement (délègue à InventoryService).
- DashboardController : KPIs + séries graphiques (30 jours).
- ReportController : génération PDF/Excel selon `?format=` (délègue à ReportService).

Pour chaque controller : Form Requests, API Resources, Policy.

Routes associées : déclaration dans `routes/api.php` pour ses endpoints — avec les middlewares de rôle appropriés (`magasinier` pour movements/transfers, `gestionnaire` pour validate/adjust).

Tests Postman — stocks & mouvements : GET stocks (filtres), POST mouvement entrée OK, POST sortie OK, POST sortie refusée (stock insuffisant → 422), vérification du recalcul du disponible.

Tests Postman — transferts : cycle complet brouillon → reçu (vérification stocks à chaque étape), annulation après validation (vérification ré-entrée source).

Tests Postman — inventaires : ouverture → saisie → ajustement (vérification mouvements d'écart générés + stocks mis à jour).

Tests Postman — rapports : vérification Content-Type PDF et Excel.

Frontend (inchangé) : pages produits + stocks, formulaire mouvement (contrôle disponible temps réel), wizard transfert, pages inventaires.

Hiba — Développeuse Frontend

Composants partagés : AppLayout, Sidebar (navigation par rôle), DataTable réutilisable (tri, pagination, recherche), StockBadge, StatusBadge, ConfirmDialog, ProtectedRoute + AuthContext.

Features : LoginPage + hook useAuth, pages entrepôts, catégories et utilisateurs (listes + modales), DashboardPage (KpiCard + 3 graphiques Recharts).

Youssef — Développeur Frontend

Transferts : TransferListPage (filtres statut), TransferFormPage (wizard multi-étapes), TransferDetailPage (actions valider / réceptionner / annuler).

Inventaires : InventoryListPage, InventorySessionPage (saisie quantité réelle + mise en évidence des écarts), InventoryAdjustPage.

Autres : ReportsPage (type + format + téléchargement blob), ActivityLogPage (timeline filtrée).

Dépendance critique à gérer dès le jour 2 : l'ordre de merge des migrations doit respecter les clés étrangères — `warehouses` → `categories` → `products` → `stocks` → `transfers` → `stock_movements` → `inventories`. Youssef (frontend transferts/inventaires) et Abdelhamid (backend des mêmes modules) doivent se synchroniser sur les contrats d'API avant d'implémenter.

Convention Postman partagée : chaque développeur backend crée sa collection Postman et l'exporte dans le dossier `/postman/` du dépôt Git. Toutes les collections utilisent une variable d'environnement `{{base_url}}` (ex. `http://localhost:8000/api/v1`) et une variable `{{token}}` renseignée après le login. Chaque requête doit documenter : le corps JSON envoyé, le code HTTP attendu, et les champs clés de la réponse à vérifier. Les cas d'erreur (401, 403, 404, 422) sont aussi couverts.

6. Diagramme de Gantt (11 → 30 juin 2026)

Le planning couvre les **14 jours ouvrés** entre le 11 et le 30 juin 2026 (weekends exclus, indiqués par les pointillés verticaux). Le dernier jour (lundi 30) est un sprint de clôture : tests croisés de toute l'équipe, corrections, documentation et déploiement.

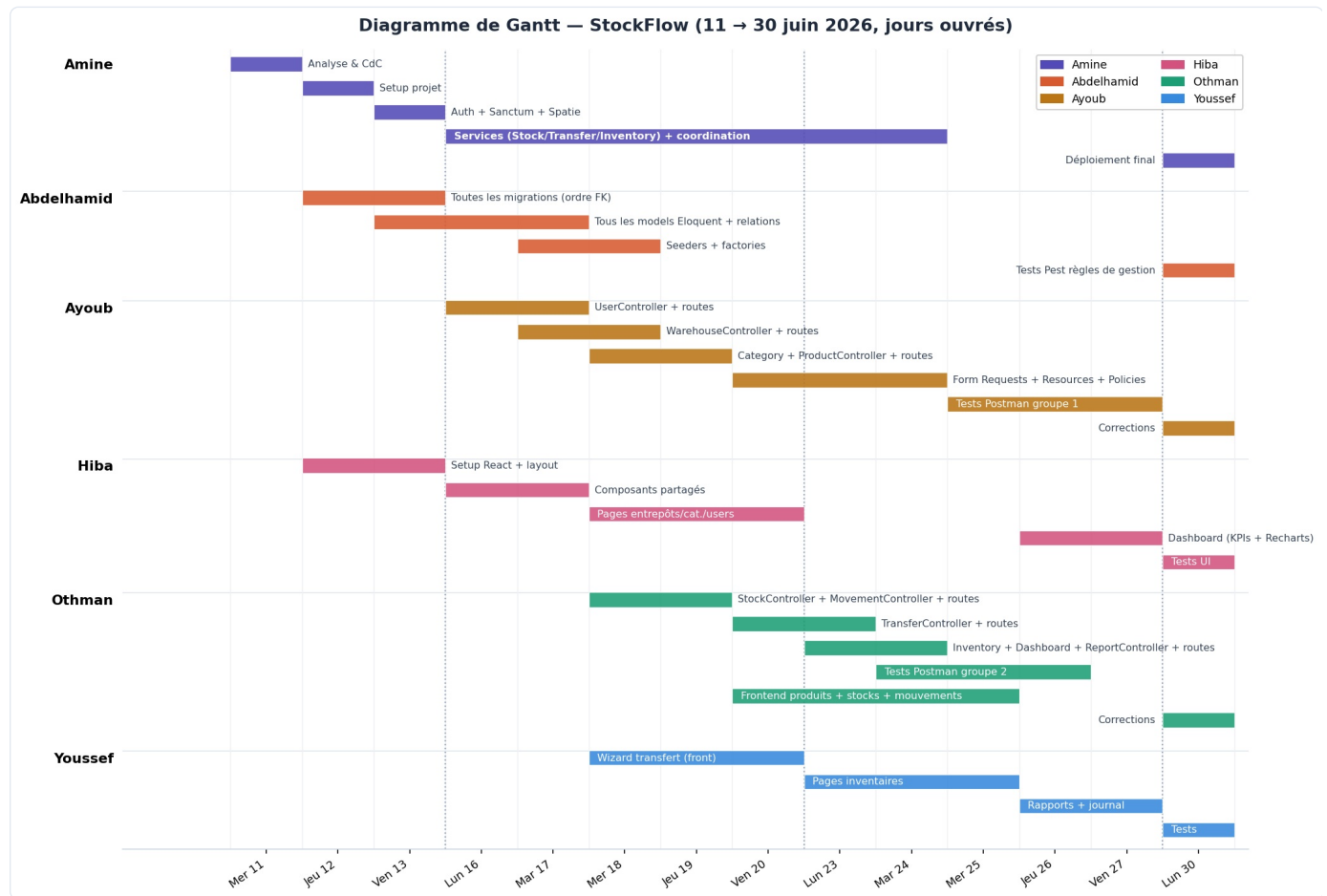


Figure 1 — Diagramme de Gantt par membre. Les barres violettes longues d'Amine correspondent à la période de développement du StockService en parallèle de la coordination et des revues de code.

6.1 Jalons clés

Date	Jalon	Critère de validation
Ven. 13 juin	Socle technique prêt	Auth fonctionnelle + rôles + layout React + premières migrations mergées.
Ven. 20 juin	Catalogue complet	CRUD entrepôts/catégories/produits opérationnel de bout en bout (API + UI).
Mer. 25 juin	Cœur métier livré	Stocks, mouvements transactionnels et cycle de transfert complets.
Ven. 27 juin	Fonctionnalités gelées	Inventaires, dashboard, rapports PDF/Excel terminés — feature freeze.
Lun. 30 juin	Livraison	Tests croisés passés, application déployée en ligne, documentation remise.